DataBase에서 가장 중요한 점은 **"동시성"**이다. 이 **"동시성"**에는 두가지 의미가 있는데,

- **Transaction**: 동시에 동작되어야 하는 작업
  (ex. 송금시 각 계좌에 변경사항 적용)
- **Concurrency Control**: 동시에 DB에 접근해야 하는 작업
  (ex. 공연 좌석 예약시 같은 DB에 여러 사람이 접근해야 함)

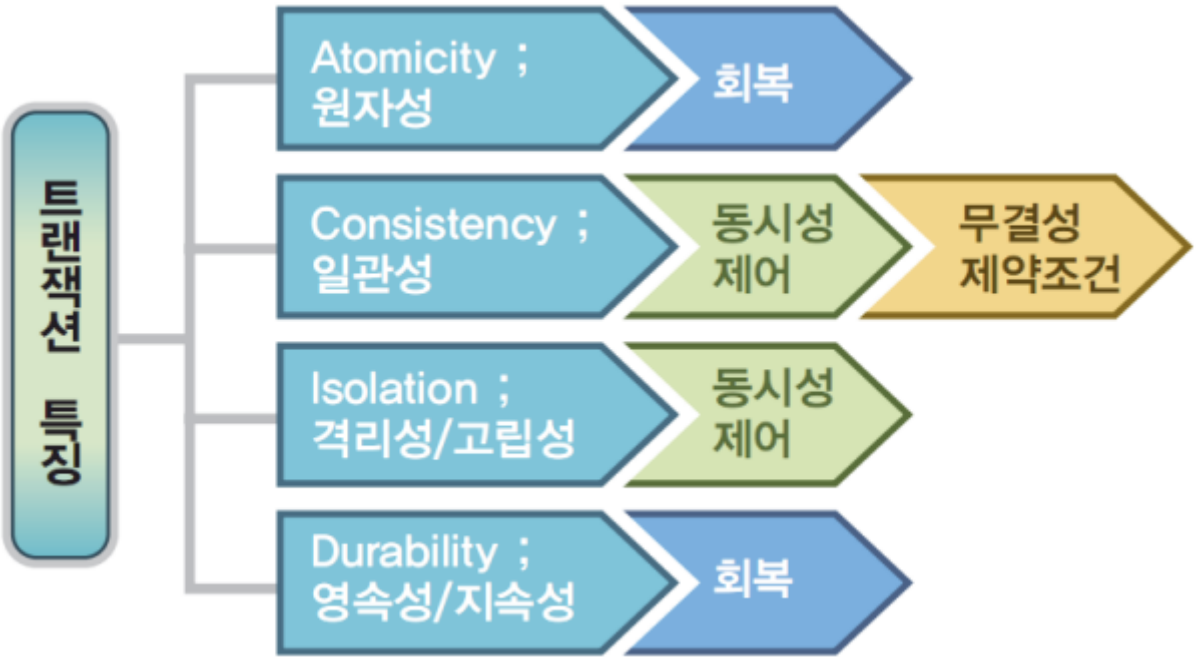
즉, 이 두 작업을 오류없이 동작하도록 하는 것이 중요하다.

Transaction

예를들어, 계좌이체나 좌석예약 같은 작업들을 생각해보면, 서로의 작업이 서로에게 영향을 미칠 수 있다는 것을 알 수 있다.
따라서 이를 어떻게 제어하는지가 중요하다.

이를 위해서는 "더이상 분할이 불가능한 업무처리의 단위", 즉 Transaction이 필요하다.

DataBase는 Transaction에서 반드시 갖춰야 할 주요 성질은 다음과 같은데 이를 위한 다양한 기능들을 활용하게 된다.

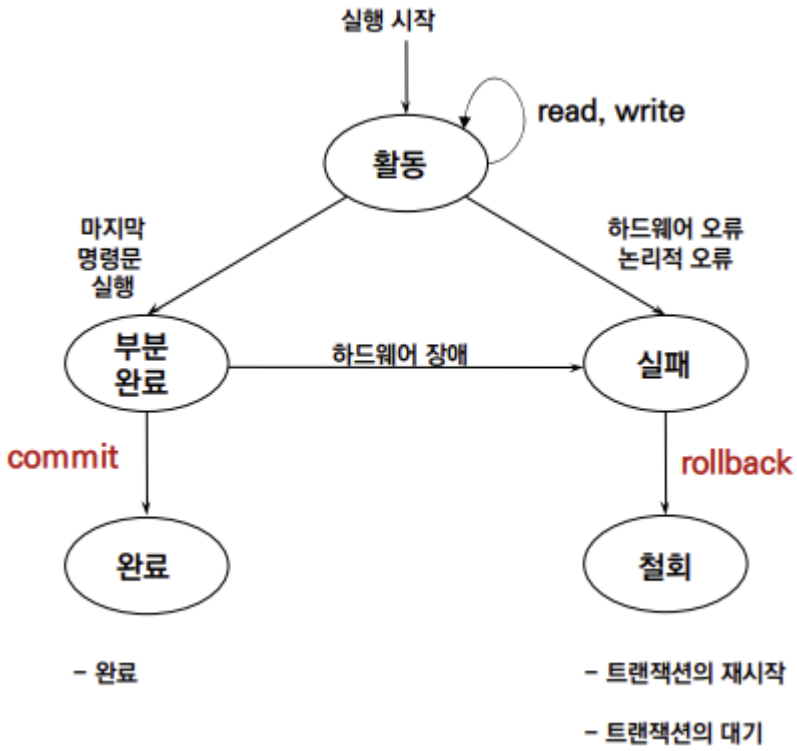


> **ACID**
>
> - **Atomicity**(*원자성*)
> : Transaction과 관련된 작업들이 실행되다가 중단되지 않음을 보장
> - **Consistency**(*일관성*)
> : Transaction이 이전과 이후의 DataBase의 상태는 모두 DataBase의 제약이나 Rule을 만족함을 보장
> - **Isolation**(*격리성*)
> : Transaction시 다른 Transaction의 연산 작업이 끼어들지 못함을 보장

> - **Durability**(*영속성*)
> : 반영(Commit)이 완료된 Transaction의 내용은 영원히 적용됨
>
> 이를 위해서 DataBase는 위와 같은 기능들을 활용하게 된다.

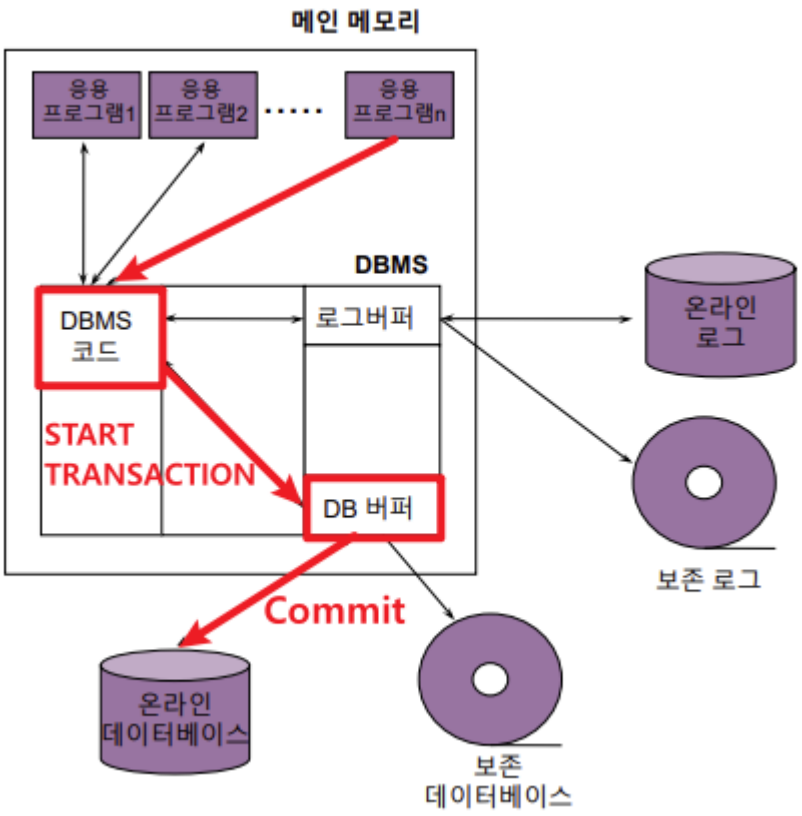
---

1. Commit & Rollback



Database에는 **Atomicity**, **Consistency**, **Durability**를 보장하기 위한 Commit과 Rollback기능이 존재한다.

1) Commit

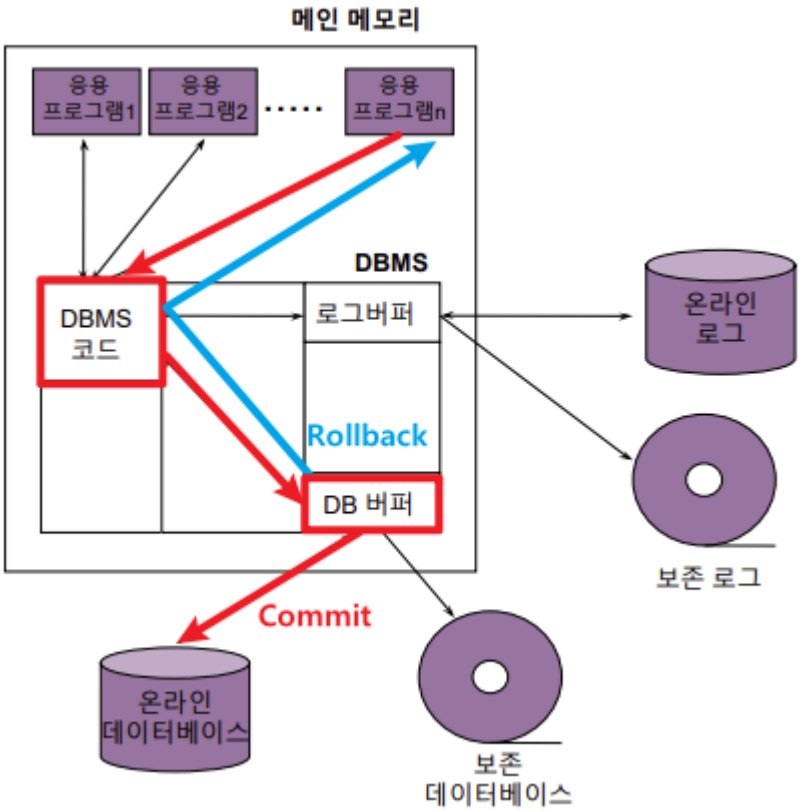


> 변경내용이 영구적으로 Database에 반영되도록 하는 기능을 의미한다.
>
> Insert나 Delete, Update와 같은 SQL 문장을 여러 번 수행할 때,
> DataBase는 우선 그 결과를 내부적으로 버퍼에 저장하고, 하나의 SQL문장이 끝날 때마다 이 결과를 Commit해 영구적으로 DataBase에 반영하게 된다.
>
> > 즉, Commit을 활용하면 **논리적인 연산 단위를 재정의** 할 수 있는데,
> > 이는 계좌이체나, 좌석예약과 같은 시스템을 구축할 때 반드시 필요하다.
>
> ---
>
> **MySQL**
>
> MySQL에는 Autocommit이라는 기능이 있는데, 이로인해 우리가 어떤 명령어를 입력하든 자동으로 Commit이 수행되게 된다.
>
> 따라서 Transaction을 위해서는 이 autocommit 기능을 꺼주고 Transaction기능을 켜야한다.
>
> - **Transaction tools**
> - `START TRANSACTION;`
> : `COMMIT`, `ROLLBACK`이 나올 때까지 실행되는 모든 SQL추적
> - `COMMIT`
> : `START TRANSACTION`이후부터 현재까지의 모든 코드 실행결과를 DB에 반영
> - `ROLLBACK`
> : `START TRANSACTION`실행 전 상태로 DB의 상태를 되돌림
>
> ---
>
> ex. 계좌이체 Procedure

```
SET autocommit=0;
START TRANSACTION;
  UPDATE account
  SET money = money+ 1000000$;
  WHERE Name="UI-JIN";

  UPDATE account
  SET money = money- 1000000$;
  WHERE Name="JU-YUNG";
COMMIT;
```

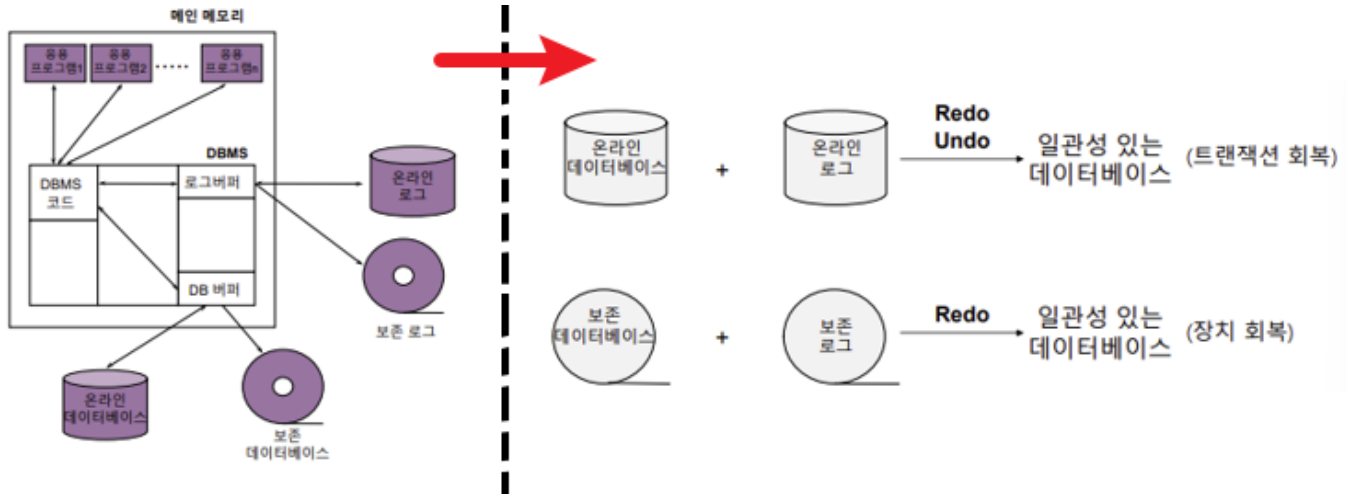
2) Rollback



> 바로 이전에 Commit이 일어났던 상태로 Database의 버퍼 상태를 복구하는 것을 의미한다.
>
> > 한번 Commit이 한번 되면 그 이전상태로는 Rollback이 불가능하기 때문에
> > 적절한 위치에서 Commit을 하는 것이 중요하다.
>
> 이는 뒤의 Undo작업과 같다.
>
> *(참고) 이때 DDL 문은 Rollback 대상이 되지 않는다는 점을 유의해야 한다.*

---

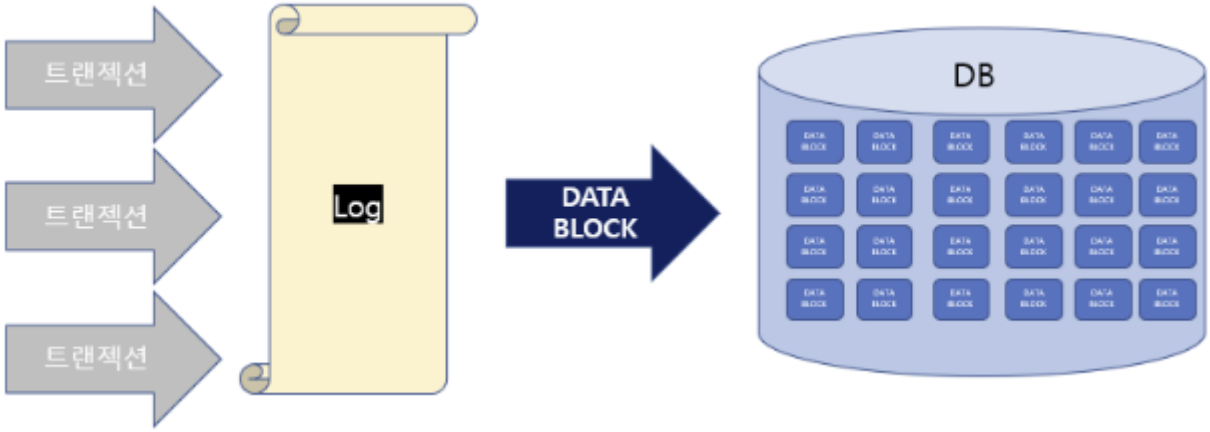
2. Redo & Undo



데이터베이스가 실행 도중 장애로 인해 손상되었다면 이를 복구하는 방법 또한 필요하다.

- 장애유형
  - Transaction defect(논리오류 및 데이터 불량)
  - System Defect(하드웨어 오동작)
  - Media Defect(디스크 고장)

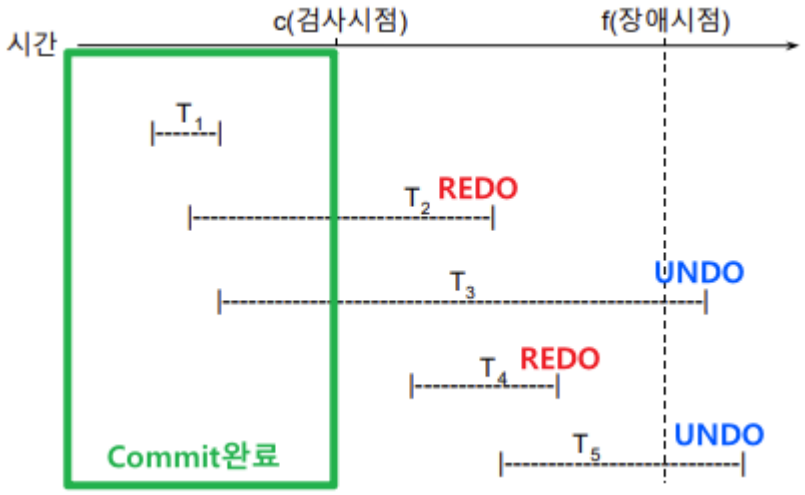
1) LogData



> 위와 같이 결함으로 인해 데이터베이스가 손상되었을 경우 이를 복구하기 위해서는 Log Data가 반드시 필요하다.
>
> ---
>
> **WAL(Write-Ahead Logging) 이론**
>
> DB는 Commit이 발생하면 바로 DataBase 서버에 변경 사항을 바로 반영하지 않는다.
>
> DB는 Sequential한 Log에 이러한 변경 사항을 적어 DB버퍼에 보관하다가 어느정도 데이터가 차게되면 Block으로 만들어 하드디스크에 Write하게 된다.
>
> 이로인해 얻을 수 있는 이점은 다음과 같다.
>
> - IO가 자주 발생하지 않기 때문에 DB의 성능을 올릴 수 있다.
> - Log에 먼저 적기 때문에 누가 조회를 하더라도 같은 Data를 보여줄 수 있다. (**Consistency**)
> - 서버가 다운되더라도 이미 Log에 먼저 기입하였기 때문에 원자성이 보장된다. (**Atomicity**)
>
> ---

> **Log 종류**
>
> - Error Log
> - General Log
> - Binary Log
> - Slow Query Log
> - ...
>
> *(참고: MySQL의 경우 my.ini파일을 통해 로그 설정을 할 수 있다.)*

2) Redo & Undo



> - REDO: Commit이 된 것을 다시 실행
> - UNDO: Transaction은 시작되었지만 Commit이 되지 않은 것을 취소
>
> ---
>
> **즉시 갱신 회복**
>
> **자연 갱신 회복**

---

3. Concurency Control

먼저 동시접속자에 의해 Concurrency Control이 필요한 경우 다음과 같은 문제들이 발생한다.

	트랜잭션1	트랜잭션2	발생하는 문제들	동시접근
case 1	read	read	문제없음	허용
case 2	read	write	• Dirty read • Non-repeatable read • Phantom read	허용 또는 불가 선택
case 3	write	write	• Lost Update • Inconsistency • Cascading rollback	lock을 이용하여 제한적으로 허용

이를 해결하기 위해 Transaction고립 Level을 설정할 수 있는데 MySQL의 경우 다음과 같은 4개의 Level이 존재한다.

- READ UNCOMMITTED
- READ-COMMITTED
- REPEATABLE READ
- SERIALIZABLE

각 단계별로 해결 가능한 문제점이 다르지만, 해결할 수 있는 문제점이 많을수록 수행 시간이 길어지기 때문에 DB의 사용 목적에 알맞는 Transaction 설계가 필요하다.

1) 문제점(1)-(READ + WRITE)

case 2	read	write	• Dirty read • Non-repeatable read • Phantom read	허용 또는 불가 선택

> **Dirty Read**
>
> | time | Transaction1 | Transaction2 |
> |---|---|---|
> | 1 | read(A)=5 | |
> | 2 | write(A)=10 | |
> | 3 | | read(A)=5 |
> | 4 | commit | |
>
> *(A=10을 읽을거라고 생각하여 읽었는데, T1이 완료된 상태가 아니기 때문에 A=5로 읽게됨)*
>
> > 원인: **write(t1, A) → read(t2, A)**
> >
> > Transaction이 완료되지 않은, 즉 Commit되지 않는 상황을 다른 Transaction에서 읽을 수 있는 현상

> **Non-Repeatable Read**
>
> | time | Transaction1 | Transaction2 |
> |---|---|---|
> | 1 | read(A)=5 | |
> | 2 | | write(A)=10 |
> | 3 | | commit |
> | 4 | read(A)=10 | |
>
> > 원인: **read(t1, A) → write(t2, A) → read(t1, A)**
> >
> > 같은 data에 대해 read를 2번했는데 두 값의 Inconsistency가 발생하는 현상
>
> ---
>
> **Pantom Read**
>
> | time | Transaction1 | Transaction2 |
> |---|---|---|
> | 1 | read(A)=[5] | |
> | 2 | | insert(A)=[5,8] |
> | 3 | | commit |
> | 4 | read(A)=[5,8] | |
>
> > 원인: **read(t1, A) → write(t2, A) → read(t1, A)**
> >
> > 같은 Relation에 대해 read를 2번했는데 추가된 값이 나타나는 현상

2) 문제점(2)-(WRITE + WRITE)

case 3	write	write	• Lost Update • Inconsistency • Cascading rollback	lock을 이용하여 제한적으로 허용

> **Lost Update**
>
> | time | Transaction1 | Transaction2 |
> |---|---|---|
> | 1 | write(A)=10 | |
> | 2 | | write(A)=20 |
> | 3 | | commit |
> | 4 | commit | |
>
> > 원인: **write(t1, A) → write(t2, A)**

> write를 한 값이 다른 Transaction에 의해 덮어씌워져 무효화되는 현상
>
> *(이 현상은 탐지 불가능(undetectable) 하기 때문에 치명적이다)*

Inconsistency

time	Transaction1	Transaction2
1	write(sum)+=A	
2		write(B)=0
3		commit
4	write(sum)+=B	
5	commit	

(A=10, B=20 일 때, Transaction1 은 sum=30 을 예상했으나 결과는 sum=10 이 됨)

> 원인: **write(t2, B) → write(t1, B)**
>
> write하기 전 다른 Transaction에 의해 값이 덮어씌워져 Transaction의 결과에 영향을 끼치는 현상

Cascading Rollback

time	Transaction1	Transaction2
1	read(A)=5	
2	write(A)=10	
3		write(A)=20
4		commit
5	rollback	

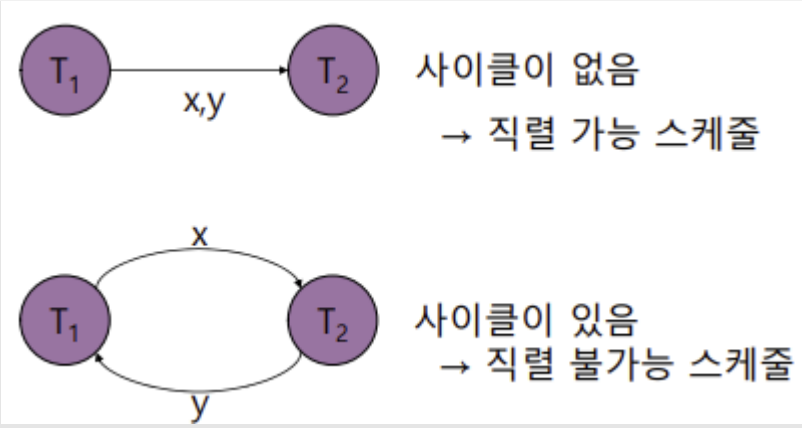
> 원인: **write(t1, A) → write(t2, A)**
>
> 작업내용을 rollback하기 전 다른 Transaction에 의해 commit되어 DB를 rollback할 수 없게 되는 현상

(참고)
: t1 rollback(time4) → t2 commit(time5) 할 경우,
t2 에 있는 버퍼의 내용을 기록하는 것이기 때문에 t2 는 성공적으로 동작한다.)

---

3) 해결책(1)-Serializability

	Type of Violation		
Isolation Level	Dirty Read	Nonrepeatable Read	Phantom
READ UNCOMMITTED	Yes	Yes	Yes
READ COMMITTED	No	Yes	Yes
REPEATABLE READ	No	No	Yes ← MySQL은 no (MVCC때문)
SERIALIZABLE	No	No	No

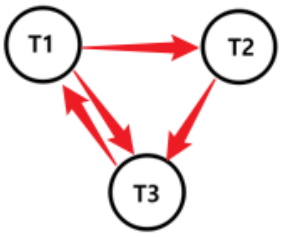
> 위와 같은 문제들은 모두 여러 Transaction을 병렬로 처리했기 때문에 발생하는 현상이다.
>
> 위에서 분석한 Transaction의 원인들을 살펴보면 다음과 같은 경우로 요약된다.
>
> 
>
>
> - **Write(T1, x) → Read(T2, x)**
> - **Read(T2, x) → Write(T1, x)**
> - **Write(T1, x) → Write(T2, x)**
>
> **<u>를 간선으로 나타내었을 때, Cycle이 없는 Schedule일 경우 Serializable하다.</u>**

따라서 위와 같은 구조로 병렬화된 Transaction Schedule이 아닐 경우, 문제없이 Transaction이 동작이 가능하다는 것이다.

즉, 이는 병렬화된 Transaction Schedule임에도 Serializable하게 동작이 가능하다는 것임을 알 수 있다.

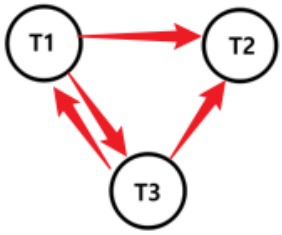
직렬 가능성 검사 예시

$R_{T1}(X) \rightarrow R_{T3}(X) \rightarrow W_{T1}(X) \rightarrow R_{T2}(X) \rightarrow W_{T3}(X)$



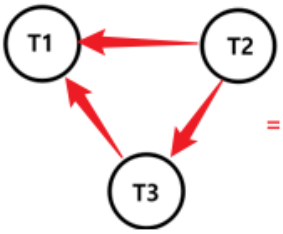
X

$R_{T1}(X) \rightarrow R_{T3}(X) \rightarrow W_{T3}(X) \rightarrow W_{T1}(X) \rightarrow R_{T2}(X)$



X

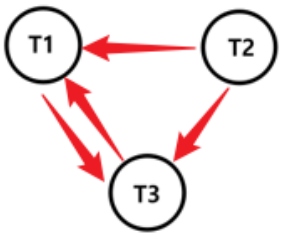
$R_{T3}(X) \rightarrow R_{T2}(X) \rightarrow W_{T3}(X) \rightarrow R_{T1}(X) \rightarrow W_{T1}(X)$



== T2→T3→T1

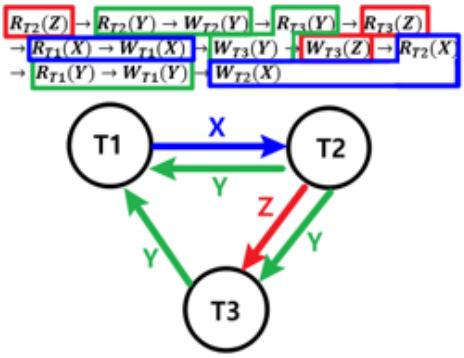
O

$R_{T3}(X) \rightarrow R_{T2}(X) \rightarrow R_{T1}(X) \rightarrow W_{T3}(X) \rightarrow W_{T1}(X)$



X

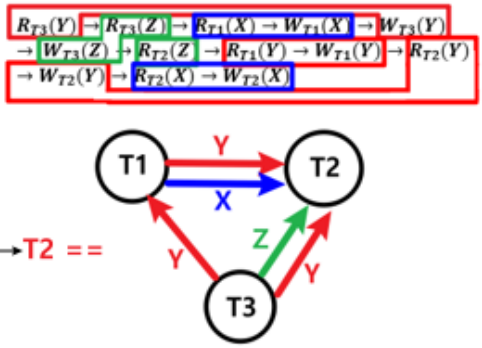
$R_{T2}(Z) \rightarrow R_{T2}(Y) \rightarrow W_{T2}(Y) \rightarrow R_{T3}(Y) \rightarrow R_{T3}(Z)$
$\rightarrow R_{T1}(X) \rightarrow W_{T1}(X) \rightarrow W_{T3}(Y) \rightarrow W_{T3}(Z) \rightarrow R_{T2}(X)$
$\rightarrow R_{T1}(Y) \rightarrow W_{T1}(Y) \rightarrow W_{T2}(X)$



X

O

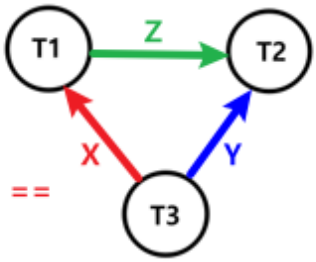
$R_{T3}(Y) \rightarrow R_{T3}(Z) \rightarrow R_{T1}(X) \rightarrow W_{T1}(X) \rightarrow W_{T3}(Y)$
$\rightarrow W_{T3}(Z) \rightarrow R_{T2}(Z) \rightarrow R_{T1}(Y) \rightarrow W_{T1}(Y) \rightarrow R_{T2}(Y)$
$\rightarrow W_{T2}(Y) \rightarrow R_{T2}(X) \rightarrow W_{T2}(X)$



T3→T1→T2 ==

$$R_{T1}(X) \rightarrow R_{T2}(Z) \rightarrow R_{T1}(Z) \rightarrow R_{T3}(X) \rightarrow R_{T3}(Y)$$
$$\rightarrow W_{T1}(X) \rightarrow W_{T3}(Y) \rightarrow R_{T2}(Y) \rightarrow W_{T2}(Z) \rightarrow W_{T2}(Y)$$

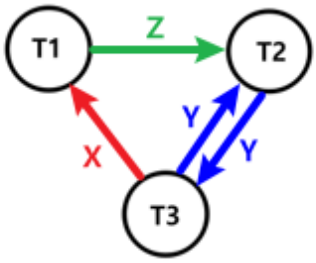
T3⟶T1⟶T2 ==



O

X

$$R_{T1}(X) \rightarrow R_{T2}(Z) \rightarrow R_{T3}(X) \rightarrow R_{T1}(Z) \rightarrow R_{T2}(Y)$$
$$\rightarrow R_{T3}(Y) \rightarrow W_{T1}(X) \rightarrow W_{T2}(Z) \rightarrow W_{T3}(Y) \rightarrow W_{T2}(Y)$$



4) 해결책(2)-2PLP

위의 Serializable은 Transaction이 계속해서 들어오거나, 문제가 복잡해질 경우 검사하는데 어려움이 존재한다.

따라서 이 Serializable을 검사하지 않고 Serializable을 보장하는 방법이 필요한데 이 방법에는 다음과 같은 방법들이 있다.

- 2PLP
- MVCC(Multi Version Concurrency Control)
- Optimistic Validation
- Snapshot Isolation

Locking

<u>상호 배제(독점 제어)를 위해 다른 Transaction에서 현재 데이터에 접근하는 것을 막는 것</u>

- **Locking모드**
  - Lock-S(공용 Lock)
    : read연산에 한하여 접근 허용
  - Lock-X(전용 Lock)
    : read, write연산 모두 불가능
- **Shared Locking Protocol**
  - 다른 Transaction에 의해 Lock이 되어 있지 않은 경우
    - read(x)
      : **<u>lock-S(x)</u>** 또는 **<u>lock-X(x)</u>**를 반드시 실행해야 함
    - write(x)
      : **<u>lock-X(x)</u>**를 반드시 실행해야 함

- 다른 Transaction에 의해 Lock이 되어 있는 경우

T_i \ T_j	S	X
S	T	F
X	F	F

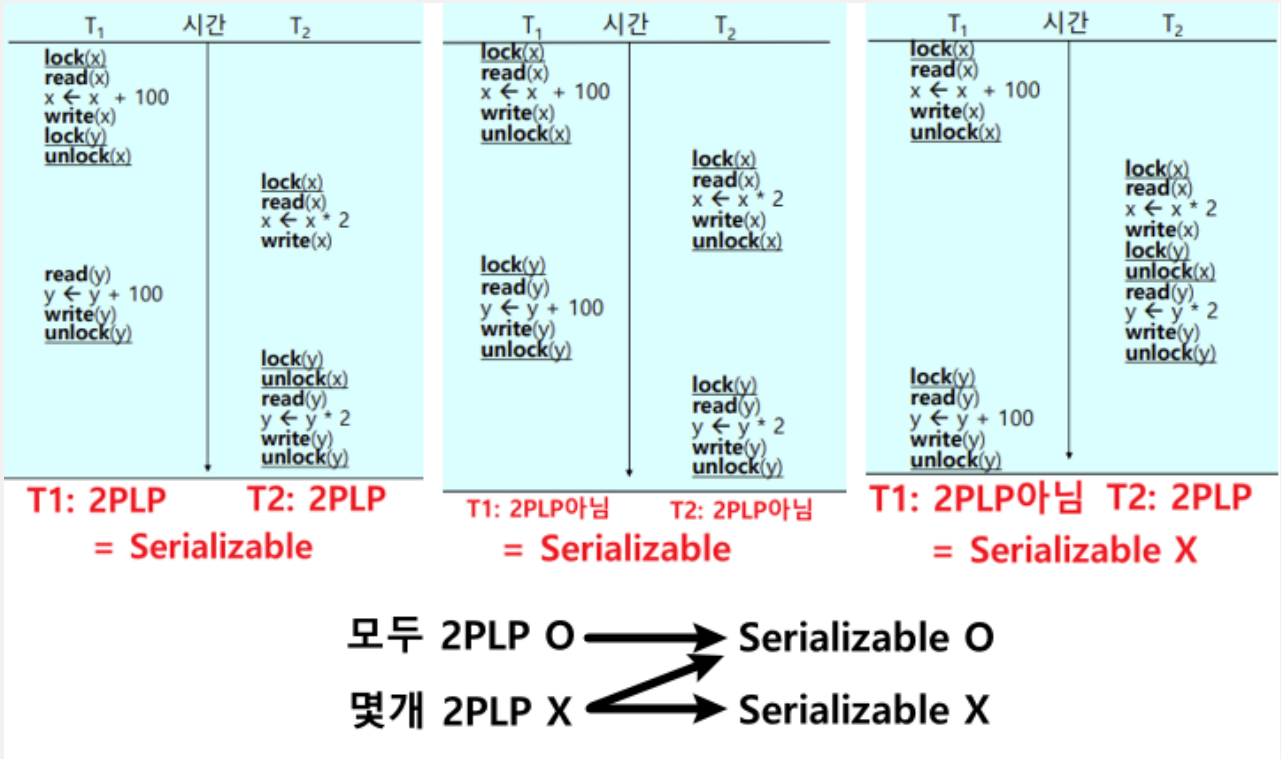
T : 접근 허용
F : 대기

  - 내 Locking과 양립 가능한 경우
    : lock 가능
  - 불가능한 경우
    : 기다려야 함
- Transaction T가 실행한 Lock(x)에 대해서는 T가 **Unlock(x)**를 통해 종료해야 함
- Transaction은 자기가 걸지 않은 Lock(x)에 대해서는 **Unlock(x)를 실행하지 못함**

> 하지만 이 Locking Protocol만 가지고는 Serializable을 보장할 수 없다.

2PLP(2-Phase Locking Protocol)

> Transaction Schedule내의 모든 Transaction들이 **2PLP**를 준수한다고 하면 해당 Schedule은 Serializable이 보장된다.
>
> - **2PLP**: 다음의 2 Phase로만 구성된 Transaction
> - **Growing Phase**
> : Locing만 수행하는 단계(중간에 Unlock금지)
> - **Shrinking Phase**
> : UnLocing만 수행하는 단계(중간에 Lock금지)



- **모든 Transaction이 2PLP → Serializable 보장**
- **모든 Transaction이 2PLP → Serializable 할 수도 있음**

> **Dead Lock**
>
> 2PLP의 경우 Lock의 위치를 잘못 선정할 경우 Dead Lock상황이 발생할 수 있다.
>
> | time | Transaction1 | Transaction2 |
> |---|---|---|
> | 1 | lock(A) | |
> | 2 | read(A) | |
> | 3 | | lock(B) |
> | 4 | | read(B) |
> | 5 | lock(B) | |
> | 6 | | lock(A) |
> | 7 | unlock(A) | |
> | 8 | | unlock(B) |
>
> - Transaction1: time5에서 Transaction2의 unlock(B)를 기다림
> - Transaction2: time6에서 Transaction1의 unlock(A)를 기다림
>
> → 두 Transaction 모두 상대방의 실행되지 않을 unlock을 기다리게 됨
>
> *회피방법에는 여러가지가 있고 이는 인터넷 참조*

위와 같이 여러 Serializable하도록 만드는 방법이 있지만,
요즘에는 대부분 MVCC(Multi Version Concurrency Control)이라는 방법을 사용함